

ELECTRAGUARD

Electricity Theft Detection & Risk Analytics System

TECHNICAL SPECIFICATION & SYSTEM DESIGN SPECIFICATION

Document Type: System Architecture & Design Document (SADD)

Target Platforms: Streamlit Cloud & GitHub Pages (Dual Deployment)

Authors: Antigravity AI (Advanced Agentic Coding)

Date: June 2026

Database: Google Cloud Firestore (Firebase Integration)

1. System Architecture Overview

ElectraGuard features a hybrid presentation and database layout split into a dual-deployment model. This allows the app to serve two key functions: lightweight static HTML client-side evaluation, and dynamic python-based server-side ML pipelines with persistent storage.

1.1 Dual-Deployment Topology

A. Static Single-Page App (GitHub Pages)

A lightweight front-end deployed on GitHub Pages using vanilla HTML, CSS, and Javascript. It includes a client-side statistical engine (`detection.js`) and a TensorFlow.js implementation of the LSTM model. This is used for sandbox evaluations and offline client-side computation.

B. Server-Side Analytics Portal (Streamlit Cloud)

The primary enterprise application, built in Python and deployed on Streamlit Cloud. This layer handles high-throughput file parsing, database management via the Firebase Admin SDK, and interactive chart rendering using Plotly. Since Streamlit Cloud operates under a strict 1GB memory limit, its ML pipeline is optimized using Scikit-Learn instead of heavy TensorFlow runtimes.

Component Layer	GitHub Pages (Static)	Streamlit Cloud (Dynamic)
Presentation	HTML5 + Vanilla CSS (<code>styles.css</code>)	Streamlit Widgets + Custom CSS Injection
Analytics Engine	Javascript anomaly detection (<code>detection.js</code>)	Python stats engine (<code>theft_detection.py</code>)
Machine Learning	TensorFlow.js LSTM model	Scikit-Learn MLPClassifier model
Database / Persistence	None (Client-side memory only)	Google Cloud Firestore (via Admin SDK)

Table 1.1: Component breakdown comparison between static and dynamic app runtimes.

2. Theft Detection Engine Design

ElectraGuard implements a hybrid detection model modeled after Kocaman & Tümen (2020) 'Detection of electricity theft using data processing and LSTM method'. The scoring logic fuses population-based statistical anomaly flags (60% weight) with sequence-based neural network classification probabilities (40% weight).

2.1 Statistical Anomaly Detectors

The Python engine (`theft_detection.py`) auto-maps arbitrary Excel/CSV columns and calculates 6 independent statistical indicators:

- 1. Z-Score Anomaly (15 pts):** Flags consumption totals deviating > 2 standard deviations from the population mean.
- 2. IQR Outlier (15 pts):** Identifies low-consumption values using interquartile range thresholds: $val < Q1 - 1.5 * IQR$.
- 3. Billing Ratio Anomaly (10 pts):** Flags consumers whose billing-to-consumption ratio is $< 50\%$ of the population average.
- 4. Load Overload Anomaly (10 pts):** Identifies meters where measured peak load exceeds sanctioned contract demand by $> 20\%$.
- 5. Consumption Drop (10 pts):** Flags a sharp, sudden decrease ($> 50\%$) in consumption compared to the previous billing cycle.
- 6. Meter Defect flag (10 pts):** Scans text descriptions for keywords indicating faulty, bypassed, or tampered meters.

2.2 Neural Network Classification (LSTM Simulation)

To run on the Streamlit Cloud free tier (limited to 1GB RAM), TensorFlow was replaced with a Scikit-Learn MLPClassifier. The model simulates the LSTM architecture (128 node hidden layer, 64 node hidden layer, ReLU activations) and processes daily consumption sequences. It uses a 7-day sliding window to capture temporal changes in load patterns. The model is evaluated using 5-Fold Cross-Validation, and classification thresholds are optimized dynamically on the validation fold using Youden's J statistic.

3. Firebase Integration & Persistence

ElectraGuard integrates with Google Cloud Firestore to provide persistent session storage and run archives. This allows utility operators to upload a dataset once, run analytics, and view reports at any time across sessions.

3.1 Data Serialization Schema

The Firestore schema utilizes a two-tier collection structure designed to optimize reads and writes:

Runs Collection (Parent)	Consumers Subcollection (Child)
Path: <code>/runs/{run_id}</code> Stores run metadata: - <code>run_id</code>: Unique UUID/Timestamp - <code>filename</code>: Original file name - <code>timestamp</code>: Firestore server time - <code>total/critical/high/medium/low</code>: Risk counts - <code>col_map</code>: Auto-detected column indexes - <code>ts_cols</code>: Time-series column list	Path: <code>/runs/{run_id}/consumers/{consumer_id}</code> Stores individual consumer analysis results: - <code>consumer_id</code>: Unique meter ID - <code>name</code>: Consumer name - <code>consumption/billing/region/category</code>: Extracted values - <code>_risk_score</code>: Combined hybrid score (0-100) - <code>_risk_level</code>: Classified level (Low-Critical) - <code>_flags</code>: Active anomaly tags - <code>_lstm_prob</code>: ML model output (if trained)

3.2 Batched Write & Type Sanitization

Firestore limits batch write commands to 500 documents per single operation. To scale to large datasets, `firebase_db.py` splits the consumer records list and commits in batches of 400 documents. Additionally, the database module cleans pandas NaN/NaT values and converts NumPy data types (such as `np.integer`, `np.floating`, and `np.bool_`) to native Python types prior to serialization to prevent API crash errors.

3.3 Authentication Workflow

Firestore connection credentials are kept secure by storing the service account private key details in Streamlit Secrets. Locally, they reside in `.streamlit/secrets.toml` (which is excluded from Git tracking via `.gitignore`), and in production, they are pasted into the Streamlit Cloud Console. If the credentials are not configured, the application automatically disables the run history page and displays a warning badge in the sidebar without throwing any errors.